

Huffman Coding | Greedy Algo-3

Last Updated : 22 Apr, 2025



Huffman coding is a lossless data compression algorithm. The idea is to assign variable-length codes to input characters, lengths of the assigned codes are based on the frequencies of corresponding characters.

The variable-length codes assigned to input characters are [Prefix Codes](#), means the codes (bit sequences) are assigned in such a way that the code assigned to one character is not the prefix of code assigned to any other character. This is how Huffman Coding makes sure that there is no ambiguity when decoding the generated bitstream.

Let us understand prefix codes with a counter example. Let there be four characters a, b, c and d, and their corresponding variable length codes be 00, 01, 0 and 1. This coding leads to ambiguity because code assigned to c is the prefix of codes assigned to a and b. If the compressed bit stream is 0001, the decompressed output may be "cccd" or "ccb" or "acd" or "ab".

There are mainly two major parts in Huffman Coding

1. Build a Huffman Tree from input characters.
2. Traverse the Huffman Tree and assign codes to characters.

Algorithm:

The method which is used to construct optimal prefix code is called **Huffman coding**. This algorithm builds a tree in bottom up manner using a priority queue (or heap)

Steps to build Huffman Tree

Input is an array of unique characters along with their frequency of occurrences and output is Huffman Tree.

1. Create a leaf node for each unique character and build a min heap of all leaf nodes (Min Heap is used as a priority queue. The value of frequency field is used to compare two nodes in min heap. Initially, the least frequent character is at root)

2. Extract two nodes with the minimum frequency from the min heap.
3. Create a new internal node with a frequency equal to the sum of the two nodes frequencies. Make the first extracted node as its left child and the other extracted node as its right child. Add this node to the min heap.
4. Repeat steps#2 and #3 until the heap contains only one node. The remaining node is the root node and the tree is complete.

Let us understand the algorithm with an example:

<i>character</i>	<i>Frequency</i>
<i>a</i>	5
<i>b</i>	9
<i>c</i>	12
<i>d</i>	13
<i>e</i>	16
<i>f</i>	45

Step 1. Build a min heap that contains 6 nodes where each node represents root of a tree with single node.

Step 2 Extract two minimum frequency nodes from min heap. Add a new internal node with frequency $5 + 9 = 14$.

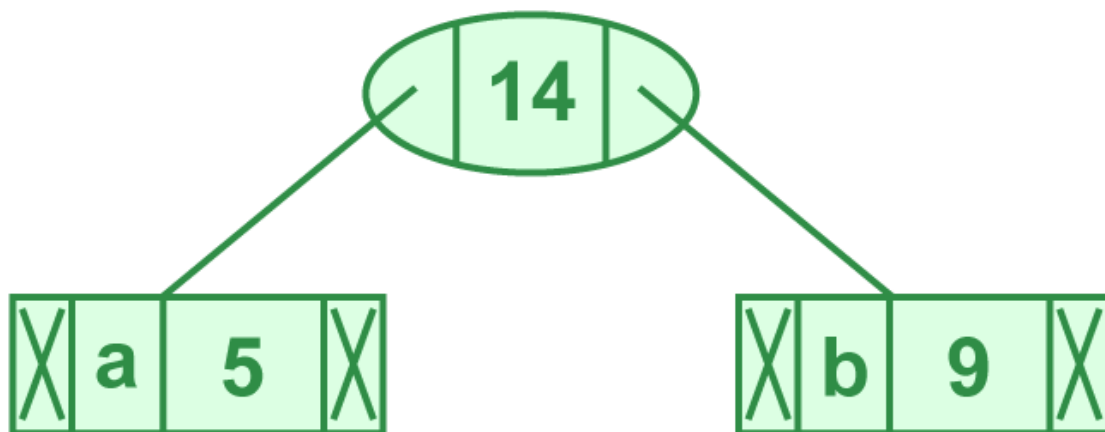


Illustration of step 2

Now min heap contains 5 nodes where 4 nodes are roots of trees with single element each, and one heap node is root of tree with 3 elements

character	Frequency
c	12
d	13
Internal Node	14
e	16
f	45

Step 3: Extract two minimum frequency nodes from heap. Add a new internal node with frequency $12 + 13 = 25$

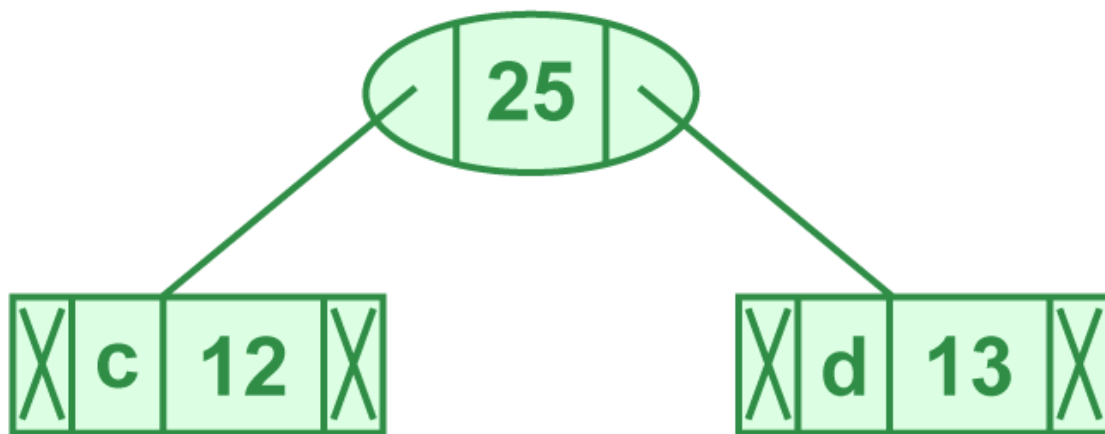


Illustration of step 3

Now min heap contains 4 nodes where 2 nodes are roots of trees with single element each, and two heap nodes are root of tree with more than one nodes

character	Frequency
Internal Node	14
e	16
Internal Node	25
f	45

Step 4: Extract two minimum frequency nodes. Add a new internal node with frequency $14 + 16 = 30$

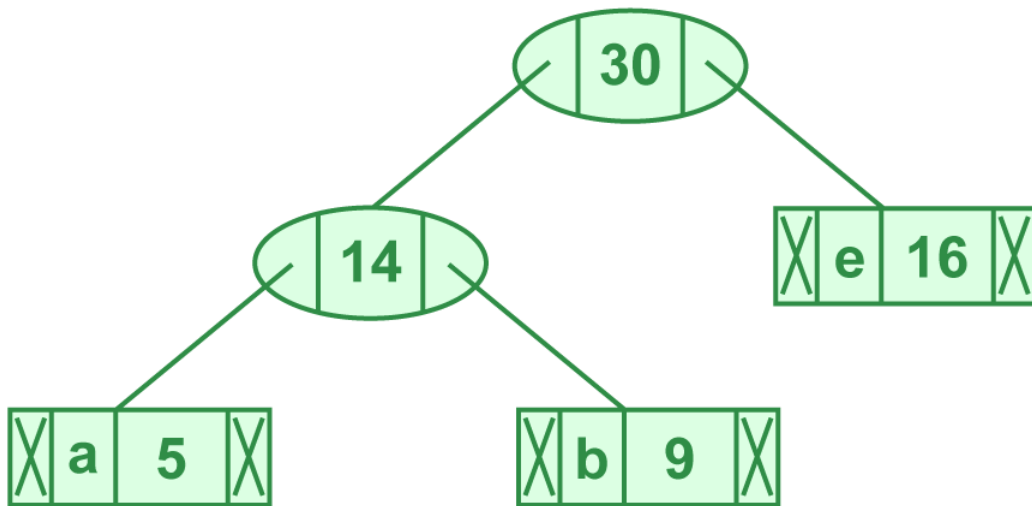


Illustration of step 4

Now min heap contains 3 nodes.

character	Frequency
Internal Node	25
Internal Node	30
f	45

Step 5: Extract two minimum frequency nodes. Add a new internal node with frequency $25 + 30 = 55$

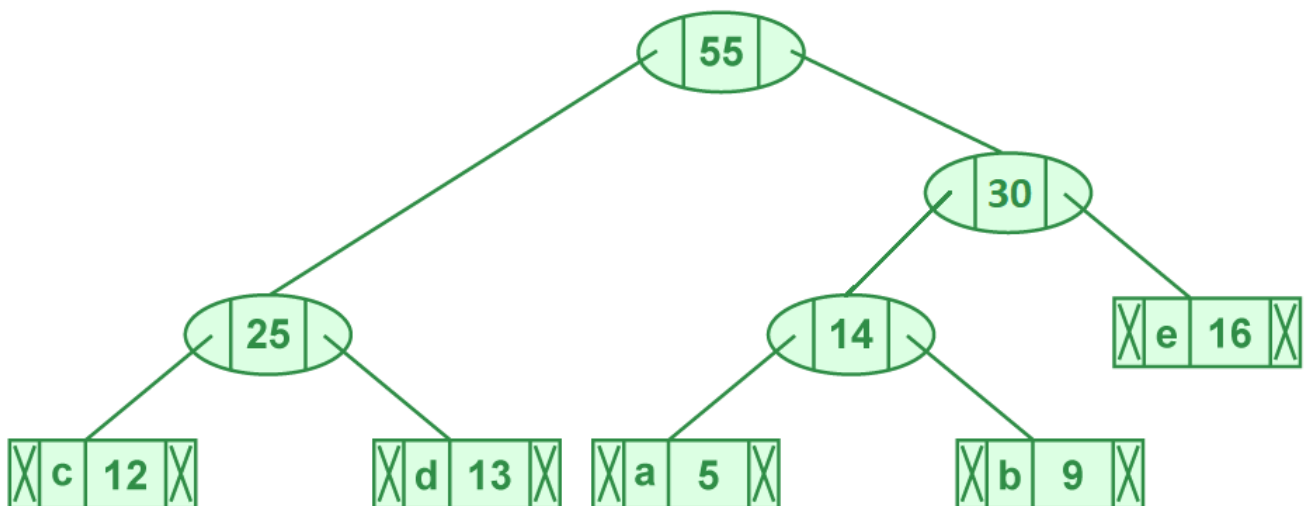


Illustration of step 5

Now min heap contains 2 nodes.

character	Frequency
f	45
Internal Node	55

Step 6: Extract two minimum frequency nodes. Add a new internal node with frequency $45 + 55 = 100$

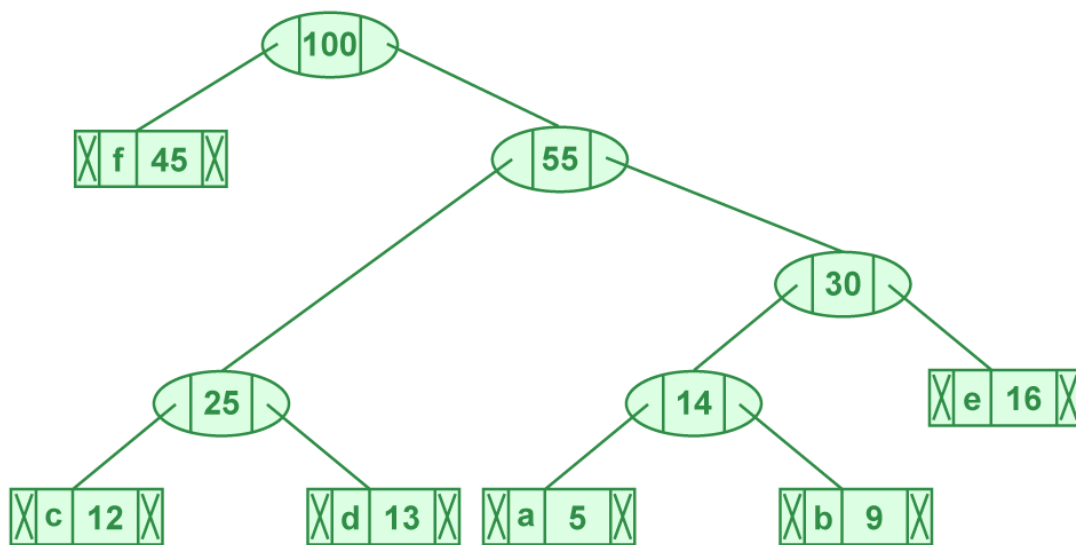


Illustration of step 6

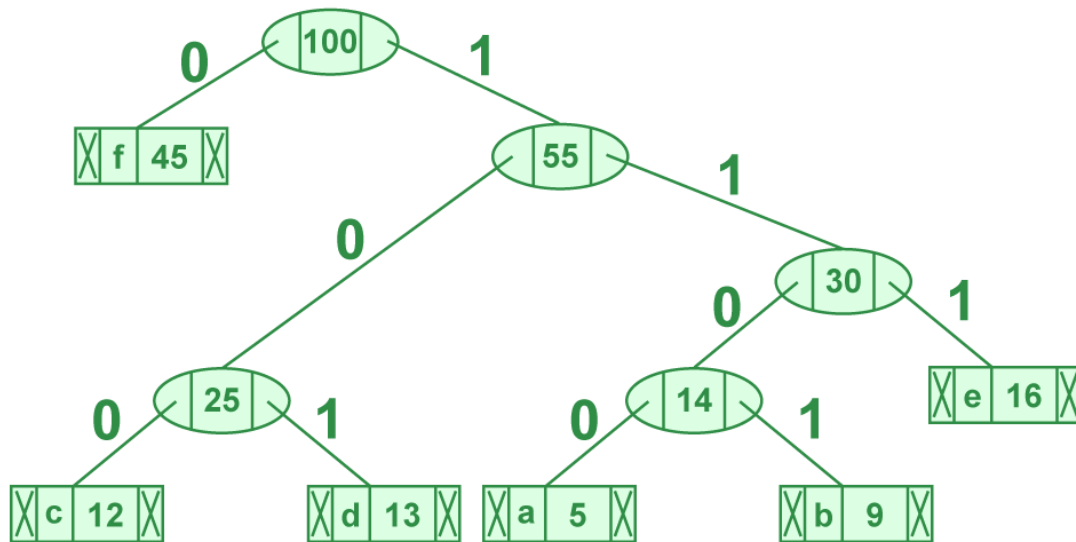
Now min heap contains only one node.

character	Frequency
Internal Node	100

Since the heap contains only one node, the algorithm stops here.

Steps to print codes from Huffman Tree:

Traverse the tree formed starting from the root. Maintain an auxiliary array. While moving to the left child, write 0 to the array. While moving to the right child, write 1 to the array. Print the array when a leaf node is encountered.



Steps to print code from HuffmanTree

The codes are as follows:

character	code-word
f	0
c	100
d	101
a	1100
b	1101
e	111

Below is the implementation of above approach:

C++ Java Python C# JavaScript

// C++ code for the above approach:

```
#include <bits/stdc++.h>
using namespace std;
```

// Class to represent huffman tree

```
class Node {
public:
    int data;
    Node *left, *right;
    Node(int x) {
        data = x;
        left = nullptr;
        right = nullptr;
    }
};
```

```

    }
};

// Custom min heap for Node class.
class Compare {
public:
    bool operator() (Node* a, Node* b) {
        return a->data > b->data;
    }
};

// Function to traverse tree in preorder
// manner and push the huffman representation
// of each character.
void preOrder(Node* root, vector<string> &ans, string curr) {
    if (root == nullptr) return;

    // Leaf node represents a character.
    if (root->left == nullptr && root->right==nullptr) {
        ans.push_back(curr);
        return;
    }

    preOrder(root->left, ans, curr + '0');
    preOrder(root->right, ans, curr + '1');
}

vector<string> huffmanCodes(string s,vector<int> freq) {

    int n = s.length();

    // Min heap for node class.
    priority_queue<Node*, vector<Node*>, Compare> pq;
    for (int i=0; i<n; i++) {
        Node* tmp = new Node(freq[i]);
        pq.push(tmp);
    }

    // Construct huffman tree.
    while (pq.size()>=2) {

        // Left node

```

```

        Node* l = pq.top();
        pq.pop();

        // Right node
        Node* r = pq.top();
        pq.pop();

        Node* newNode = new Node(l->data + r->data);
        newNode->left = l;
        newNode->right = r;

        pq.push(newNode);
    }

    Node* root = pq.top();
    vector<string> ans;
    preOrder(root, ans, "");
    return ans;
}

int main() {
    string s = "abcdef";
    vector<int> freq = {5, 9, 12, 13, 16, 45};
    vector<string> ans = huffmanCodes(s, freq);
    for (int i=0; i< ans.size(); i++) {
        cout << ans[i] << " ";
    }

    return 0;
}

```

Output

```
0 100 101 1100 1101 111
```

Time complexity: $O(n \log n)$ where n is the number of unique characters

Space complexity :- $O(n)$

Applications of Huffman Coding:

1. They are used for transmitting fax and text.
2. They are used by conventional compression formats like PKZIP, GZIP, etc.

3. Multimedia codecs like JPEG, PNG, and MP3 use Huffman encoding(to be more precise the prefix codes).

It is useful in cases where there is a series of frequently occurring characters.

Huffman Coding

[Comment](#)[More info](#) ▼[Advertise with us](#)[Next Article](#) >[Huffman Coding using Priority Queue](#)

Similar Reads

Huffman Coding | Greedy Algo-3

Huffman coding is a lossless data compression algorithm. The idea is to assign variable-length codes to input characters, lengths of the assigned codes are based on the frequencies of corresponding...

🕒 12 min read

Huffman Coding using Priority Queue

Prerequisite: Greedy Algorithms | Set 3 (Huffman Coding), `priority_queue::push()` and `priority_queue::pop()` in C++ STL Given a char array `ch[]` and frequency of each character as `freq[]`. The...

🕒 12 min read

Canonical Huffman Coding

Huffman Coding is a lossless data compression algorithm where each character in the data is assigned a variable length prefix code. The least frequent character gets the largest code and the most frequent on...

🕒 11 min read

Huffman Decoding

We have discussed Huffman Encoding in a previous post. In this post, decoding is discussed. Examples: Input Data: AAAAAABCCCCCDDDEEEEFrequencies: A: 6, B: 1, C: 6, D: 2, E: 5 Encoded Data:...

🕒 15 min read

Efficient Huffman Coding for Sorted Input | Greedy Algo-4

We recommend to read following post as a prerequisite for this.Greedy Algorithms | Set 3 (Huffman Coding) Time complexity of the algorithm discussed in above post is $O(n \log n)$. If we know that the give...

🕒 15+ min read

Text File Compression And Decompression Using Huffman Coding

Text files can be compressed to make them smaller and faster to send, and unzipping files on devices has a low overhead. The process of encoding involves changing the representation of a file so that the...

🕒 14 min read

Image Compression using Huffman Coding

Huffman coding is one of the basic compression methods, that have proven useful in image and video compression standards. When applying Huffman encoding technique on an Image, the source symbols...

🕒 15+ min read

Adaptive Huffman Coding And Decoding

Prerequisite: Huffman Coding, Huffman Decoding Adaptive Huffman Coding is also known as Dynamic Huffman Coding. The implementation is done using Vitter Algorithm. Encoding Adaptive Huffman codin...

🕒 7 min read
